Binary Search Tree Reviewer

HOA 9 · Trees · C++ (`baiSerTree` in `bst.h`) · includes time complexity analysis

1. Quick Reference

Item	Rule
BST property	For every node: all keys in the **left** subtree are **smaller**; all keys in the **right** subtree are **larger**
Max children per node	2 (left and right)
Insert	Compare with the current node. Smaller: go left. Larger: go right. Empty spot: place the new node there
Duplicates	Ignored in `bst.h` (neither < nor > is true, so nothing happens)
Pre-order	Root → Left → Right
In-order	Left → Root → Right (**gives sorted order in a BST**)
Post-order	Left → Right → Root
Search / insert time	O(h): O(log n) when balanced, O(n) when the tree is a chain
Traversal time	O(n) for all three (every node is visited once)

Memory trick: the name says *where the root is printed*. **Pre** = root first. **In** = root in the middle. **Post** = root last. Left is always visited before right.

2. Tree Terminology

Term	Meaning
Node	One element holding a key and pointers to its children
Root	Top node with no parent
Parent / Child	A node directly above / below another node
Siblings	Nodes sharing the same parent
Leaf	Node with no children
Edge	Link between a parent and a child
Subtree	A node together with all of its descendants
Depth of a node	Number of edges from the **root down to** the node (root = 0)
Height of a node	Number of edges on the longest path from the node **down to a leaf** (leaf = 0)
Height of the tree (h)	Height of the root
Level	All nodes with the same depth

Depth vs. height: depth looks **up** to the root; height looks **down** to the deepest leaf. If your course counts nodes instead of edges, add 1 to every value.

Tree types from your HOA 9 tasks

Type	Children per node	Ordering rule	Your class
General tree	Any number	None	genTree (`firstChild` / `nextSibling` linked lists)
Binary tree	At most 2	None	(no separate class)
Binary search tree	At most 2	Left < node < Right	`baiSerTree`

Every BST is a binary tree, and every binary tree is a general tree. The reverse is not true.

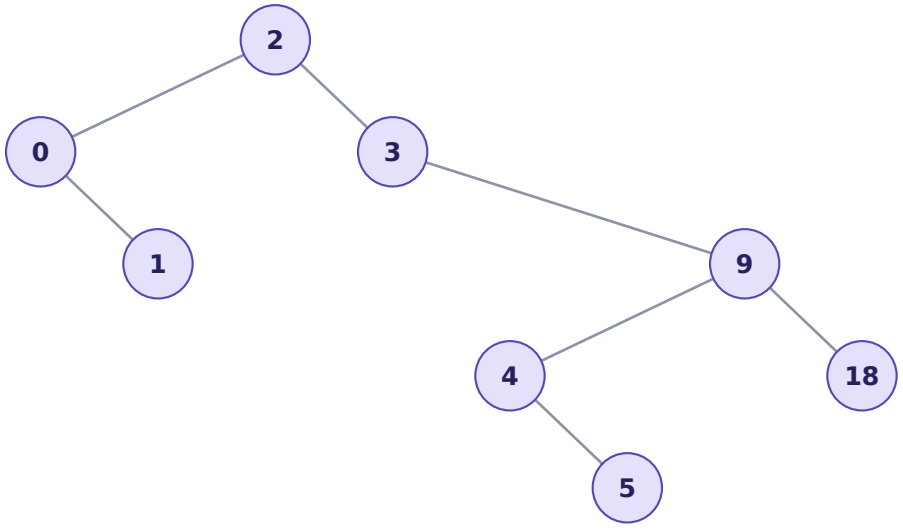
3. How Insertion Works

```
insert(value):
    if value < key:
        if left is empty  -> create new node here
        else              -> left->insert(value)      (recurse)
    else if value > key:
        if right is empty -> create new node here
        else              -> right->insert(value)     (recurse)
    else:
        do nothing                                    (duplicate)
```

4. Worked Example: build the BST from 2, 3, 9, 18, 0, 1, 4, 5

The first value (2) becomes the root through the constructor. The remaining values are inserted in order.

Step	Insert	Comparisons	Result
0	2	(root)	2 is the root
1	3	3 > 2 → right (empty)	3 is the right child of 2
2	9	9 > 2 → right; 9 > 3 → right (empty)	9 is the right child of 3
3	18	18 > 2, 18 > 3, 18 > 9 → right (empty)	18 is the right child of 9
4	0	0 < 2 → left (empty)	0 is the left child of 2
5	1	1 < 2 → left; 1 > 0 → right (empty)	1 is the right child of 0
6	4	4 > 2, 4 > 3, 4 < 9 → left (empty)	4 is the left child of 9
7	5	5 > 2, 5 > 3, 5 < 9 → left; 5 > 4 → right (empty)	5 is the right child of 4



Final tree after all insertions

Traversals of this tree

Traversal	Order rule	Result
Pre-order	Root, Left, Right	`2 0 1 3 9 4 5 18`
In-order	Left, Root, Right	`0 1 2 3 4 5 9 18`
Post-order	Left, Right, Root	`1 0 5 4 18 9 3 2`

The in-order result is **sorted ascending**. This is how you can check that a BST was built correctly.

Depth and height of every node (edge-count convention)

Node	2	0	3	1	9	4	18	5
Depth	0	1	1	2	2	3	3	4
Height	4	1	3	0	2	1	0	0

The height of the tree is 4, from the path 2 → 3 → 9 → 4 → 5.

5. Tracing a Traversal by Hand

Post-order on the subtree rooted at 9 (children 4 and 18, with 5 under 4):

Step	Call / action	Output so far
1	postOrder(9) goes to its left child: postOrder(4)	
2	postOrder(4) has no left child, so it calls postOrder(5)	
3	postOrder(5) has no children, so it prints 5	5
4	Back in postOrder(4): both sides done, so it prints 4	5 4
5	Back in postOrder(9): go right, postOrder(18) prints 18	5 4 18
6	postOrder(9) finished both subtrees, so it prints 9	5 4 18 9

Exam method: for any node, write (left subtree result) + (node) + (right subtree result) in the order the traversal demands, and work from the bottom up.

6. Time Complexity Analysis

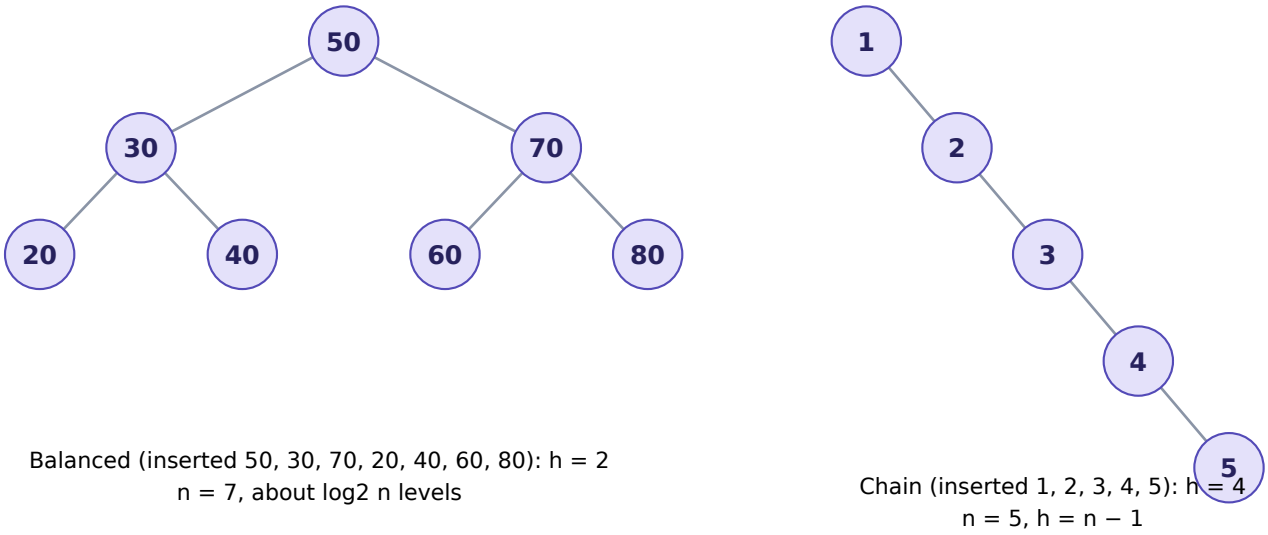
6.1 Notation

n = number of nodes in the tree. **h** = height of the tree (edges from the root to the deepest leaf). The cost of most BST operations depends on **h**, and h depends on the **shape** of the tree, which depends on the insertion order.

6.2 Why insert and search cost O(h)

Each recursive call of `insert` or `search` does a constant amount of work (one or two comparisons) and then moves **exactly one level down**. The recursion stops at the latest when it falls off a leaf. So the number of calls is at most h + 1, and the total time is O(h).

6.3 Balanced tree vs. chain



Balanced (inserted 50, 30, 70, 20, 40, 60, 80): h = 2
n = 7, about log2 n levels

Chain (inserted 1, 2, 3, 4, 5): h = 4
n = 5, h = n − 1

The same operations behave very differently depending on the shape

A binary tree of height h holds at most $2^{h+1} - 1$ nodes, so with n nodes the **smallest possible height** is floor($\log_2$ n). The **largest possible height** is n − 1 (a chain).

n (nodes)	Minimum height (balanced)	Maximum height (chain)	Max comparisons to search
7	2	6	3 (balanced) vs. 7 (chain)
15	3	14	4 vs. 15
1,000	9	999	10 vs. 1,000
1,000,000	19	999,999	20 vs. 1,000,000

The maximum number of comparisons on a root-to-leaf path is h + 1. That is why a balanced tree with a million keys needs about 20 comparisons, and a chain with the same keys needs up to a million.

6.4 Best, average, and worst case

Case	Tree shape	Height h	Insert / search
Best	Target is the root, or the tree is perfectly balanced	0 to $\log_2$ n	O(1) if the key is at the root; otherwise O(log n)

Case	Tree shape	Height h	Insert / search
Average	Keys arrive in random order (expected height is O(log n))	about 1.39 $\log_2 n$	O(log n)
Worst	Keys arrive sorted or reverse sorted, giving a chain	$n - 1$	O(n)

6.5 Complexity of each operation in baiSerTree

Operation	Best	Average	Worst	Reason
insert(value)	O(1)	O(log n)	O(n)	Walks one path from the root down; one level per call
search(value)	O(1)	O(log n)	O(n)	Same path as insert; stops early if the key is found
height()	O(n)	O(n)	O(n)	Must look at every node to find the deepest leaf
preOrder / inOrder / postOrder	O(n)	O(n)	O(n)	Each node is visited and printed exactly once
Destructor	O(n)	O(n)	O(n)	Deletes every node once (children before parent)
Constructor	O(1)	O(1)	O(1)	Creates one node

6.6 Traversal analysis

Let a traversal be T(n) on a tree with n nodes, with k nodes in the left subtree and $n - k - 1$ in the right. Each call prints one key (constant work) and recurses on the two subtrees:

```
T(n) = T(k) + T(n - k - 1) + c        T(0) = c'
```

Every node is reached by exactly one call, and every empty child causes one extra constant-time check, so the total work is about $c \cdot n + c' \cdot (n + 1)$. Therefore **T(n) = O(n)** for all three traversals, regardless of tree shape. **Space:** the recursion stack is O(h) (O(log n) balanced, O(n) for a chain).

6.7 Cost of building the whole tree

Building a BST means n inserts, and each insert costs O(current height). Counting comparisons for the two trees from section 6.3 (the root itself needs no comparison):

Tree	Comparisons per insert	Total	Growth
Balanced (50 root; 30, 70, 20, 40, 60, 80)	1 + 1 + 2 + 2 + 2 + 2	10	O(n log n)
Chain (1 root; 2, 3, 4, 5)	1 + 2 + 3 + 4	10	$O(n^2)$: $n(n - 1)/2$ comparisons

The totals are equal here only because n is tiny. For n = 1,000 a balanced build needs about 10,000 comparisons, and a chain build needs 499,500.

6.8 Worked search examples on the activity tree (2, 3, 9, 18, 0, 1, 4, 5)

Search for	Path followed	Nodes visited	Result
4	2 → 3 → 9 → 4	4	Found
5	2 → 3 → 9 → 4 → 5	5 (= h + 1)	Found
7	2 → 3 → 9 → 4 → 5 → right of 5 is empty	5	Not found
1	2 → 0 → 1	3	Found

The worst case for this tree is 5 nodes visited, which is h + 1 with h = 4. Every search follows a single root-to-leaf path, so it never visits more than h + 1 nodes.

6.9 Space complexity

Item	Space	Explanation
Storage for n keys	O(n)	One node per key: the key plus two pointers
Recursive insert / search / traversals	O(h)	One stack frame per level of the path being followed
Extra memory per operation	O(1) heap	Insert allocates one new node; traversals allocate nothing

6.10 Comparison with other structures

Structure	Search	Insert	Sorted output	Notes
Unsorted array / linked list	O(n)	O(1) at end / head	O(n log n) (sort first)	Simple; slow search
Sorted array	O(log n)	O(n)	O(n)	Binary search works, but inserting shifts elements
BST (average)	O(log n)	O(log n)	O(n) via in-order	Fast when balanced
BST (worst: chain)	O(n)	O(n)	O(n) via in-order	Behaves like a linked list
Self-balancing BST (AVL, red-black)	O(log n)	O(log n)	O(n) via in-order	Rebalances to guarantee h = O(log n)

Takeaway: a plain BST is as fast as O(log n) only if it stays balanced. Its behavior depends on the input order, and self-balancing variants such as AVL and red-black trees fix this by restructuring the tree during insertion.

6.11 Your general tree (genTree) for comparison

Operation	Time	Reason
insert(value)	O(c)	Walks the child list to its end before appending; c = number of children of that node
find(value) / findData(choice, key)	O(n) worst case	No ordering, so it may have to look at every node
preOrder / postOrder / inOrder	O(n)	Every node visited once

The BST search beats the general tree search because the ordering rule lets it discard a whole subtree at every step.

7. Fully Commented Code (bst.h)

```
//
// Created by royga on 9/17/2026.
//

#ifndef UNTITLED1_BST_H
#define UNTITLED1_BST_H

#include <iostream>

// Binary search tree. Each object is one node that also acts as the
// root of its own subtree.
template <typename T>
class baiSerTree {
private:
    T key;                 // value stored in this node
    baiSerTree<T>* left;   // subtree with SMALLER keys (nullptr if none)
    baiSerTree<T>* right;  // subtree with LARGER keys (nullptr if none)

public:
    // Constructor: creates a leaf node
    baiSerTree(T value);

    // Destructor: deleting a node deletes its whole subtree
    ~baiSerTree();

    // Insert function
    void insert(T value);

    // Traversing algorithms
    void preOrder();
    void inOrder();
    void postOrder();
};

template <typename T>
baiSerTree<T>::baiSerTree(T value) {
    key = value;
    left = nullptr;    // a new node starts with no children
    right = nullptr;
}

template <typename T>
baiSerTree<T>::~baiSerTree() {
    delete left;       // destructor runs on the left subtree (recursive)
    delete right;      // destructor runs on the right subtree (recursive)
}

// Time: O(h). One comparison and one level down per call.
template <typename T>
void baiSerTree<T>::insert(T value) {
    if (value < key) {                        // smaller: belongs on the left
        if (left == nullptr) {
            left = new baiSerTree<T>(value);  // empty spot found
        } else {
            left->insert(value);              // keep going down the left side
        }
    } else if (value > key) {                 // larger: belongs on the right
        if (right == nullptr) {
            right = new baiSerTree<T>(value);
        } else {
            right->insert(value);
        }
    }
    // value == key: duplicate, nothing happens
}

// Pre-order: Root -> Left -> Right      Time: O(n)
```

```
template <typename T>
void baiSerTree<T>::preOrder() {
    std::cout << key << " ";   // visit root first
    if (left != nullptr) {
        left->preOrder();
    }
    if (right != nullptr) {
        right->preOrder();
    }
}

// In-order: Left -> Root -> Right (sorted output in a BST)   Time: O(n)
template <typename T>
void baiSerTree<T>::inOrder() {
    if (left != nullptr) {
        left->inOrder();
    }
    std::cout << key << " ";   // printed only once, in the middle
    if (right != nullptr) {
        right->inOrder();
    }
}

// Post-order: Left -> Right -> Root     Time: O(n)
template <typename T>
void baiSerTree<T>::postOrder() {
    if (left != nullptr) {
        left->postOrder();
    }
    if (right != nullptr) {
        right->postOrder();
    }
    std::cout << key << " ";   // printed only once, at the end
}

#endif //UNTITLED1_BST_H
```

main.cpp for the worked example

```
#include <iostream>
#include "bst.h"

int main() {
    baiSerTree<int> tree(2);   // ROOT: first value

    tree.insert(3);
    tree.insert(9);
    tree.insert(18);
    tree.insert(0);
    tree.insert(1);
    tree.insert(4);
    tree.insert(5);

    std::cout << "In-order:   "; tree.inOrder();
    std::cout << "\nPre-order:  "; tree.preOrder();
    std::cout << "\nPost-order: "; tree.postOrder();
    std::cout << "\n";
    return 0;
}
```

Expected console output:

```
In-order:   0 1 2 3 4 5 9 18
Pre-order:  2 0 1 3 9 4 5 18
Post-order: 1 0 5 4 18 9 3 2
```

Extras that may be asked (not yet in your bst.h)

```
// Search: goes left or right using the BST rule.   Time: O(h)
bool search(T value) {
```

```
    if (value == key) return true;
    if (value < key)  return left != nullptr && left->search(value);
    return right != nullptr && right->search(value);
}

// Height in edges: an empty child counts as -1 so a leaf has height 0.
// Time: O(n), because every node must be checked.
int height() {
    int lh = (left == nullptr)  ? -1 : left->height();
    int rh = (right == nullptr) ? -1 : right->height();
    return 1 + (lh > rh ? lh : rh);
}
```

For the worked example, `search(4)` returns true, `search(7)` returns false, and `height()` on the root returns 4.

8. Why the Insertion Order Matters

The same values in a different order can give a different tree, but the **in-order result is always sorted**.

Insert order	Shape	Height	Pre-order	Post-order
50, 30, 70, 20, 40, 60, 80	Balanced	2	`50 30 20 40 70 60 80`	`20 40 30 60 80 70 50`
1, 2, 3, 4, 5 (already sorted)	Chain (skewed right)	4	`1 2 3 4 5`	`5 4 3 2 1`

A sorted input creates a chain, so search and insert become O(n) instead of O(log n) (see section 6).

9. Common Mistakes

Mistake	Correction
Using genTree when the task says BST	`genTree.insert` adds a **child to that node** with no comparing, so every value inserted on the root becomes a direct child of the root. Use `baiSerTree` and `#include "bst.h"`
Drawing a node with 3 children as a BST	A BST node has at most 2 children. A tree with 3 children is a general tree
Placing 3 to the left of 2	3 > 2, so 3 goes **right**
Forgetting that the first value is the root	The constructor takes the first value. Only the remaining values go through `insert`
Mixing up pre-order and post-order	Pre-order prints the root **first**. Post-order prints it **last**
Expecting duplicates to be stored	`bst.h` ignores duplicates
Saying BST search is always O(log n)	Only when the tree is balanced. A chain gives O(n)
Counting height in nodes vs. edges	Check the convention used in class. Edges: leaf = 0. Nodes: leaf = 1

10. Practice Problems

Problem 1. Insert 8, 3, 10, 1, 6, 14, 4, 7, 13 into an empty BST. Draw the tree, then give all three traversals and the height.

Problem 2. The in-order output of a BST is 4 7 9 12 15 20. Is this consistent with a valid BST? What does it tell you?

Problem 3. Which traversal prints keys in ascending order? Which one lets you delete a whole tree safely (children before parent)?

Problem 4. Insert 40, 20, 60, 10, 30, 50, 70. State the depth of node 30 and the height of node 20.

Problem 5. Predict the output if the values 5, 3, 8, 3, 5 are inserted (root 5) and inOrder() is called.

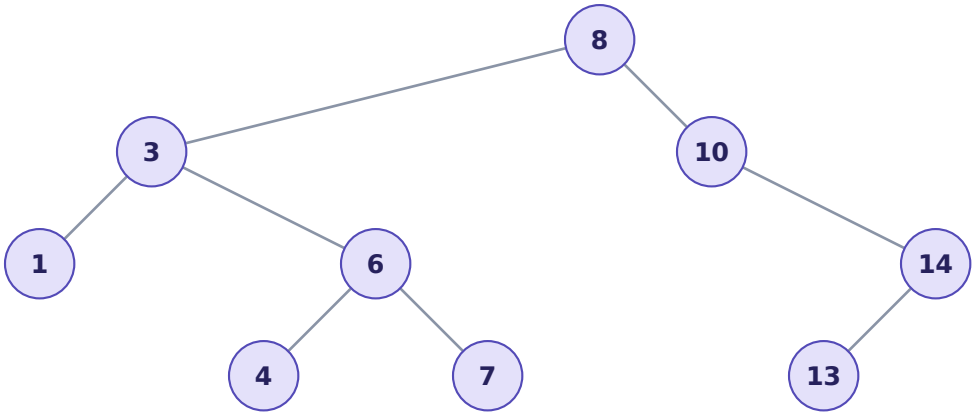
Problem 6 (complexity). What is the worst-case time to search a BST with n nodes, and what input produces it?

Problem 7 (complexity). How many comparisons does a search need, at most, in a balanced BST holding 1,000,000 keys?

Problem 8 (complexity). Inserting 1, 2, 3, ..., 100 in that order into an empty BST: what is the height, and how many comparisons does inserting the last value (100) take?

Answers

Problem 1.



Traversal	Result
Pre-order	`8 3 1 6 4 7 10 14 13`
In-order	`1 3 4 6 7 8 10 13 14`
Post-order	`1 4 7 6 3 13 14 10 8`

Height = 3 (for example 8 → 3 → 6 → 4).

Problem 2. Yes. It is sorted ascending, which is exactly what in-order gives for a valid BST. It does **not** reveal the shape of the tree, since many different BSTs have the same in-order output.

Problem 3. Ascending order: **in-order**. Safe deletion: **post-order**. The destructor in bst.h works this way.

Problem 4. The tree is balanced: 40 is the root, 20 and 60 are its children, and 10, 30, 50, 70 are the leaves. Depth of node 30 = 2. Height of node 20 = 1.

Problem 5. The duplicates 3 and 5 are ignored. The tree holds 5, 3, 8. Output: 3 5 8.

Problem 6. O(n). It happens when the input is sorted or reverse sorted, which turns the tree into a chain of height n − 1.

Problem 7. About 20 comparisons (h = 19, so at most h + 1 = 20 nodes are visited).

Problem 8. Sorted input gives a chain, so the height is 99. Inserting 100 compares with all 99 existing nodes on the way down, so 99 comparisons. Building the whole tree costs 1 + 2 + ... + 99 = 4,950 comparisons, which is $O(n^2)$.

11. Self-Check Before the Quiz

☐ I can state the BST property and use it to place any value.

☐ I can build a BST from a list, showing the comparisons at each step.

☐ I can write pre-, in-, and post-order results without looking at the code.

☐ I know the in-order result of a BST is always sorted.

☐ I can compute the depth and height of any node.

☐ I can explain why insert and search are O(h), and why h can range from log n to n − 1.

☐ I can state the time and space complexity of every operation in section 6.5.

☐ I can tell a BST from a general tree and pick the right class.